COMPTE RENDU DE TRAVAUX PRATIQUES

Réalisation d'un Feu Tricolore avec la carte STM32F407 Discovery

Enseignant : Atef Ftirich
Étudiantes : Wissal Baabou – Mbarka Ben Jalloul – Takwa Chwatt
Classe : LISI 3A
Logiciel : Keil uVision (STM32)

1. Objectif du TP

L'objectif de ce TP est de réaliser un **feu tricolore simple** (Rouge → Jaune → Vert → Jaune) en utilisant la carte **STM32F407 Discovery**. Le programme doit configurer les broches PA5, PA6 et PA7 en sortie numérique, allumer successivement les LEDs avec des temporisations adaptées, et tourner en boucle infinie.

2. Matériel utilisé

Élément	Qté	Description
Carte STM32F407 Discovery	1	Microcontrôleur STM32F407VGT6
Breadboard	1	Planche d'essai
LED Rouge	1	Connectée sur PA5
LED Jaune	1	Connectée sur PA6
LED Verte	1	Connectée sur PA7
Résistances 220 Ω / 330 Ω	3	Protection des LEDs
Fils de connexion	—	Jumpers mâle-mâle

3. Schéma de câblage

Broche STM32	Connexion	Composant
PA5	→ Anode + Résistance → GND	LED Rouge
PA6	→ Anode + Résistance → GND	LED Jaune
PA7	→ Anode + Résistance → GND	LED Verte
GND	→ Rail GND breadboard	Masse commune

4. Problématique

Lors de la réalisation de ce TP, nous avons rencontré plusieurs difficultés :

- **Le code ne fonctionnait pas** au début : les LEDs ne s'allumaient pas du tout.
- **La simulation dans Keil** ne marchait pas (le simulateur ne gère pas bien les GPIO STM32F4).
- Certaines LEDs restaient éteintes ou s'allumaient faiblement après le câblage.
- Le délai (`delay`) était trop court, rendant le changement des LEDs trop rapide.

Ces problèmes nous ont obligés à analyser le code, vérifier le câblage et tester plusieurs versions.

5. Résolution des problèmes

Problème	Cause	Solution
LEDs éteintes	Horloge GPIOA non activée	Ajout de `RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN`
Simulation Keil échoue	Simulateur limité pour STM32F4	Test direct sur la carte réelle
LED faible / éteinte	Polarité inversée ou résistance trop grande	Vérification du sens + résistances 220 Ω
Changement trop rapide	Valeur de delay trop petite	Augmentation : 9 000 000 et 4 000 000
Plusieurs LEDs allumées	Mauvaise utilisation de ODR	`GPIOA->ODR = (1U << x)` pour une seule LED

Après ces corrections, le code a fonctionné correctement sur la carte STM32F407 Discovery.

6. Développement – Explication détaillée du code

6.1 Inclusion et fonction delay

`#include "stm32f407xx.h"` — Fichier d'en-tête contenant les définitions des registres du STM32F407. La fonction `delay` est une temporisation busy-wait. Le mot-clé `volatile` empêche l'optimisation du compilateur. `__NOP()` consomme des cycles d'horloge sans rien faire.

6.2 Activation de l'horloge

`RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;` — Avant d'utiliser un périphérique, il faut activer son horloge. Sans cette ligne, le port GPIOA reste désactivé et les écritures n'ont aucun effet.

6.3 Configuration des broches en sortie

Le registre `MODER` configure chaque broche sur 2 bits : 00 = Entrée, 01 = Sortie, 10 = AF, 11 = Analogique. On efface d'abord les bits concernés, puis on force la valeur 01 pour PA5, PA6 et PA7.

6.4 Boucle principale

La boucle `while(1)` exécute indéfiniment la séquence. L'écriture `GPIOA->ODR = (1U << x)` allume uniquement la LED choisie et éteint automatiquement les autres.

7. Code final complet

```c
#include "stm32f407xx.h"

void delay(volatile uint32_t time)
{
    while (time--)
    {
        __NOP();
    }
}

int main(void)
{
    /* Activation de l'horloge du port GPIOA */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

```

```
    /* Configuration de PA5, PA6 et PA7 en sortie */
    GPIOA->MODER &= ~(3U << 10); // Clear PA5
    GPIOA->MODER &= ~(3U << 12); // Clear PA6
    GPIOA->MODER &= ~(3U << 14); // Clear PA7

    GPIOA->MODER |= (1U << 10); // PA5 = Rouge (Output)
    GPIOA->MODER |= (1U << 12); // PA6 = Jaune (Output)
    GPIOA->MODER |= (1U << 14); // PA7 = Vert (Output)

    while (1)
    {
        /* Rouge */
        GPIOA->ODR = (1U << 5);
        delay(9000000);

        /* Jaune */
        GPIOA->ODR = (1U << 6);
        delay(4000000);

        /* Vert */
        GPIOA->ODR = (1U << 7);
        delay(9000000);

        /* Jaune */
        GPIOA->ODR = (1U << 6);
        delay(4000000);
    }
}
```

7.1 Inclusion du fichier d'en-tête

```
#include "stm32f407xx.h"
```

Cette directive du préprocesseur inclut le fichier d'en-tête correspondant au microcontrôleur STM32F407. Ce fichier permet d'utiliser les noms symboliques des périphériques, des registres et de leurs bits, par exemple :

- `RCC` : périphérique de gestion des horloges et des réinitialisations ;
- `GPIOA` : port d'entrée/sortie A ;
- `RCC_AHB1ENR_GPIOAEN` : masque du bit d'activation de l'horloge de GPIOA.

L'utilisation de ces noms évite d'écrire directement les adresses mémoire des registres.

7.2 Déclaration de la fonction de temporisation

```
void delay(volatile uint32_t time)
```

Cette ligne définit une fonction appelée `delay`.

- `void` indique que la fonction ne renvoie aucune valeur.
- `uint32_t` est un type entier non signé de 32 bits.
- `time` représente le compteur utilisé pour réaliser l'attente.
- `volatile` impose au compilateur de conserver les accès à ce compteur. Cela ne désactive pas toutes les optimisations et ne garantit pas une durée précise.

```
{
```

L'accolade ouvrante marque le début du corps de la fonction.

```
while (time--)
```

Cette boucle se répète tant que la valeur de `time` avant décrémentation est différente de zéro. L'opérateur `--` placé après la variable réalise une post-décrémentation : la valeur actuelle est d'abord utilisée pour le test, puis le compteur diminue d'une unité.

Ainsi, pour `delay(4000000)`, le corps de la boucle est exécuté 4 000 000 de fois.

```
{
    __NOP();
}
```

Les accolades délimitent le corps de la boucle.

L'instruction `__NOP()` signifie *No Operation*. Elle n'effectue aucune opération utile sur les données du programme.

Elle est exécutée à chaque passage dans la boucle. La temporisation résulte de l'ensemble des instructions exécutées : accès au compteur, décrémentation, test, branchement et instruction `__NOP()`.

```
}
```

Cette accolade termine la fonction `delay`.

Remarque : cette temporisation est une attente active : le processeur reste occupé pendant toute sa durée. Le paramètre `time` représente un nombre d'itérations, et non un nombre de millisecondes.

7.3 Début de la fonction principale

```
int main(void)
{
```

La fonction `main` contient le programme principal. Dans un projet embarqué classique, elle est appelée après l'exécution du code de démarrage et les initialisations prévues par le projet.

- `int` indique un type de retour entier.
- `void`, entre parenthèses, indique que la fonction ne reçoit aucun argument.
- L'accolade ouvrante marque le début de la fonction.

Dans ce programme, la boucle infinie empêche d'atteindre la fin de `main` pendant le fonctionnement normal.

7.4 Activation de l'horloge du port GPIOA

```
/* Activation de l'horloge du port GPIOA */
```

Cette ligne est un commentaire. Elle facilite la lecture du programme mais ne produit aucune instruction exécutable.

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
```

Cette instruction active l'horloge du port GPIOA.

- `RCC` désigne le contrôleur des horloges.
- `->` permet d'accéder à un membre de la structure représentant le périphérique.
- `AHB1ENR` est le registre d'activation des horloges des périphériques du bus AHB1.

— `RCC_AHB1ENR_GPIOAEN` sélectionne le bit d'activation de GPIOA.

— `|=` réalise un OU binaire suivi d'une affectation : le bit sélectionné est mis à 1 sans modifier les autres bits du registre.

Cette activation doit précéder la configuration et l'utilisation du port GPIOA.

7.5 Principe de configuration du registre MODER

Le registre `MODER` contient deux bits de configuration pour chaque broche du port. Pour une broche de numéro n, les bits concernés sont $2n$ et $2n + 1$.

Valeur	Mode de fonctionnement
00	Entrée numérique
01	Sortie numérique
10	Fonction alternative
11	Mode analogique

Les broches utilisées correspondent donc aux champs suivants :

— PA5 : bits 11 et 10 ;

— PA6 : bits 13 et 12 ;

— PA7 : bits 15 et 14.

La configuration s'effectue en deux étapes : effacer les deux bits de chaque champ, puis écrire la valeur 01.

7.6 Effacement des bits de configuration

```
GPIOA->MODER &= ~(3U << 10);
```

Cette instruction met à zéro les bits 11 et 10, qui correspondent à PA5.

— `3U` représente la valeur non signée 3, soit 11 en binaire.

— `<< 10` décale ces deux bits de 10 positions vers la gauche.

— `~` inverse les bits du masque.

— `&=` réalise un ET binaire suivi d'une affectation.

Le masque obtenu efface uniquement les deux bits de PA5, sans modifier les autres champs du registre.

```
GPIOA->MODER &= ~(3U << 12);
```

Cette ligne applique le même principe à PA6 : les bits 13 et 12 sont mis à zéro.

```
GPIOA->MODER &= ~(3U << 14);
```

Cette ligne met à zéro les bits 15 et 14, qui correspondent à PA7.

Après ces trois instructions, les champs de configuration de PA5, PA6 et PA7 valent 00. Il reste à les placer en mode sortie.

7.7 Configuration de PA5, PA6 et PA7 en sortie

```
GPIOA->MODER |= (1U << 10);
```

Cette instruction met le bit 10 à 1. Le bit 11 ayant été effacé précédemment, le champ de PA5 prend la valeur 01.

PA5 est donc configurée en sortie numérique pour commander la LED rouge.

```
GPIOA->MODER |= (1U << 12);
```

Cette instruction met le bit 12 à 1. Le champ de PA6 devient 01 : PA6 est configurée en sortie numérique pour commander la LED jaune.

```
GPIOA->MODER |= (1U << 14);
```

Cette instruction met le bit 14 à 1. Le champ de PA7 devient 01 : PA7 est configurée en sortie numérique pour commander la LED verte.

7.8 Début de la boucle infinie

```
while (1)
{
```

En langage C, une valeur différente de zéro représente une condition vraie. La condition `1` étant toujours vraie, les instructions placées dans cette boucle sont exécutées indéfiniment.

Cette boucle permet de répéter automatiquement la séquence : Rouge → Jaune → Vert → Jaune.

7.9 Allumage de la LED rouge

```
/* Rouge */
GPIOA->ODR = (1U << 5);
```

Le commentaire indique l'étape correspondant à la LED rouge. Le registre `ODR`, ou *Output Data Register*, contient les valeurs de sortie du port.

L'expression `1U << 5` crée une valeur dont seul le bit 5 vaut 1. L'affectation avec `=` écrit cette valeur dans le registre `ODR` :

- le bit de sortie de PA5 est mis à 1 ;
- les bits de sortie de PA6 et PA7 sont mis à 0 ;
- les autres bits de sortie du port A sont également mis à 0.

Avec le câblage prévu, actif à l'état haut, la LED rouge s'allume et les LEDs jaune et verte s'éteignent.

```
delay(9000000);
```

Le programme appelle la fonction de temporisation avec 9 000 000 itérations. Pendant cette attente, aucune nouvelle valeur n'est écrite dans `ODR` : la LED rouge reste donc allumée.

7.10 Premier allumage de la LED jaune

```
/* Jaune */
GPIOA->ODR = (1U << 6);
```

Cette instruction écrit une valeur dont seul le bit 6 vaut 1. PA6 passe à l'état haut, tandis que PA5 et PA7 passent à l'état bas. La LED jaune s'allume et les deux autres LEDs s'éteignent.

```
delay(4000000);
```

La LED jaune reste allumée pendant une attente de 4 000 000 itérations. Le nombre d'itérations étant inférieur à celui utilisé pour la LED rouge, cette phase est plus courte dans les mêmes conditions d'exécution.

7.11 Allumage de la LED verte

```
/* Vert */
GPIOA->ODR = (1U << 7);
```

Cette instruction met le bit 7 de `ODR` à 1 et les autres bits de sortie à 0. PA7 passe à l'état haut : la LED verte s'allume. Les LEDs rouge et jaune s'éteignent.

```
delay(9000000);
```

La LED verte reste allumée pendant une attente de 9 000 000 itérations, comme la LED rouge.

7.12 Deuxième allumage de la LED jaune

```
/* Jaune */
GPIOA->ODR = (1U << 6);
```

Cette instruction allume à nouveau la LED jaune et éteint la LED verte. Cette deuxième phase jaune termine la séquence avant le retour à la LED rouge.

```
delay(4000000);
```

Le programme conserve cet état pendant une attente de 4 000 000 itérations.

7.13 Fin de la boucle et de la fonction principale

```
    }
}
```

La première accolade ferme le corps de la boucle `while(1)`. Le programme revient alors au début de cette boucle et recommence par l'allumage de la LED rouge.

La deuxième accolade ferme la fonction `main`. Elle n'est pas atteinte pendant le fonctionnement normal, puisque la boucle est infinie.

7.14 Précautions et limites du programme

— **Temporisation non calibrée :** les valeurs 9000000 et 4000000 ne correspondent pas directement à des secondes. La durée réelle dépend notamment de la fréquence du processeur, du compilateur et de ses options d'optimisation. Les durées annoncées dans les résultats doivent donc être présentées comme mesurées ou estimées.

— **Écriture sur tout le port :** l'instruction `GPIOA->ODR = ...` remplace l'ensemble des bits de sortie du port A. Elle convient à ce montage simple si aucune autre sortie du même port ne doit conserver son état. Dans un projet plus complet, le registre `BSRR` permet de mettre à 1 ou à 0 les seules broches sélectionnées, sans modifier les autres.

— **Hypothèse de câblage :** les explications supposent que chaque LED est connectée entre sa broche GPIO et la masse, avec une résistance en série. Dans cette configuration, un état haut commande l'allumage de la LED.

8. Résultats et observations

Après correction du code et vérification du câblage, les résultats obtenus sont les suivants :

Étape	LED	Broche	Durée	Résultat
1	Rouge	PA5	Longue (∼2 s)	OK – LED allumée
2	Jaune	PA6	Courte (∼1 s)	OK – LED allumée
3	Verte	PA7	Longue (∼2 s)	OK – LED allumée
4	Jaune	PA6	Courte (∼1 s)	OK – LED allumée

Le cycle se répète correctement en boucle infinie. Les LEDs sont bien protégées par les résistances. La séquence Rouge → Jaune → Vert → Jaune est clairement visible. Le montage fonctionne parfaitement sur la carte.

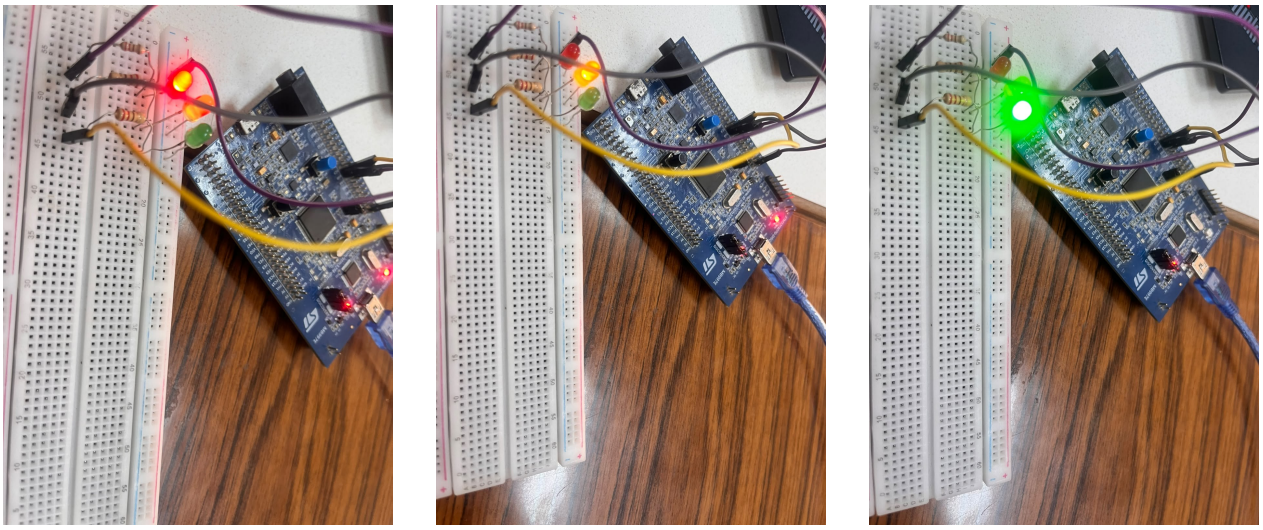


Figure 1: Montage réel sur la carte STM32F407 Discovery : LED rouge allumée (PA5, à gauche), LED jaune allumée (PA6, au centre) et LED verte allumée (PA7, à droite).

9. Conclusion

Ce TP nous a permis de mettre en pratique les notions fondamentales de la programmation embarquée sur STM32 : l'activation des horloges via le RCC, la configuration des broches GPIO en sortie, l'écriture dans le registre ODR, et la création d'une temporisation simple. Malgré les problèmes rencontrés au début (simulation Keil non fonctionnelle et code initial incorrect), nous avons réussi à identifier les erreurs, les corriger et faire fonctionner le feu tricolore correctement sur la carte réelle.

— Fin du compte rendu —